

CS722PE: REAL TIME SYSTEMS

(Professional Elective - V)

IV Year B.Tech. IT I -Sem

UNIT – II

- Introduction:
 - Real Time Operating Systems:
 - Brief History of OS,
 - Defining RTOS,
 - The Scheduler,
 - Objects,
 - Services,
 - Characteristics of RTOS,
 - Defining a Task,
 - Tasks States and Scheduling,
 - Task Operations,
 - Structure,
 - Synchronization,
 - Communication and Concurrency.
 - Defining Semaphores,
 - Operations and Use,
 - Defining Message Queue,
 - States, Content,
 - Storage,
 - Operations and Use

Unit-2 Part-1

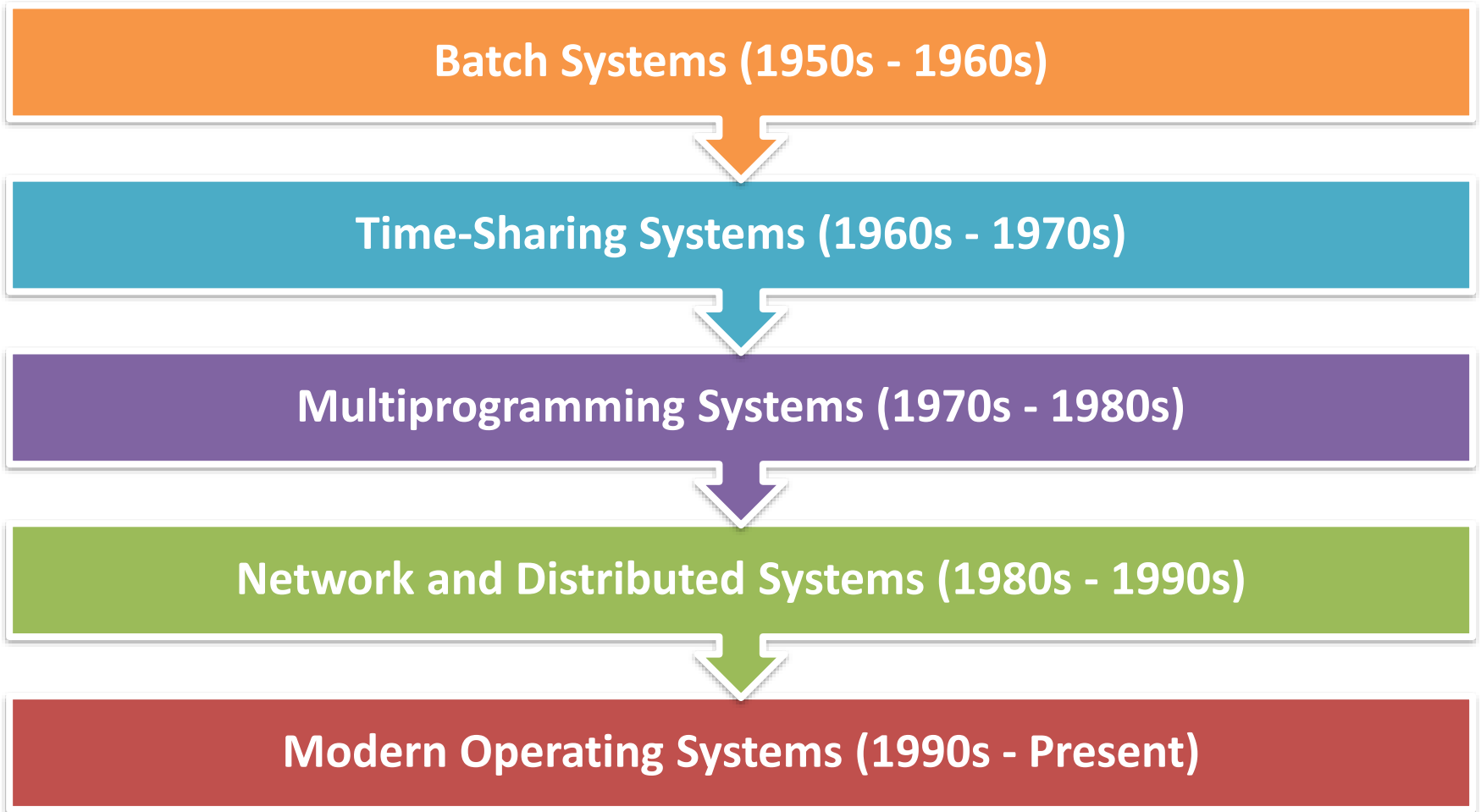
- Brief History of OS,
- Defining RTOS,
- The Scheduler,
- Objects,
- Services,
- Characteristics of RTOS

Brief History of Operating Systems

- Operating systems (OS) have evolved from simple programs to complex systems responsible for managing hardware resources, executing applications, and ensuring the overall performance and security of computing devices. Here's a summary of their history:
- **Batch Systems (1950s - 1960s):** Early computers had no OS. Programs were manually loaded and executed. The need for automation led to the development of batch processing, where jobs were executed in sequences without user interaction.
- **Time-Sharing Systems (1960s - 1970s):** Computers became more powerful, enabling multiple users to interact with them simultaneously. Time-sharing OSES were developed to allow multiple users to share the CPU by dividing processing time into small intervals (time slices).
- **Multiprogramming Systems (1970s - 1980s):** With advances in memory management, OSES could run multiple programs concurrently. This led to better CPU utilization and faster response times.
- **Network and Distributed Systems (1980s - 1990s):** With the growth of networking, OSES began to include support for sharing resources across connected computers, giving rise to distributed computing.
- **Modern Operating Systems (1990s - Present):** Modern OSES are sophisticated, multi-user, multi-tasking systems with enhanced security, graphical interfaces, and support for multiple hardware types. Examples include Windows, Linux, and macOS.

Brief History of Operating Systems

Batch Systems (1950s - 1960s)



```
graph TD; A[Batch Systems (1950s - 1960s)] --> B[Time-Sharing Systems (1960s - 1970s)]; B --> C[Multiprogramming Systems (1970s - 1980s)]; C --> D[Network and Distributed Systems (1980s - 1990s)]; D --> E[Modern Operating Systems (1990s - Present)];
```

Time-Sharing Systems (1960s - 1970s)

Multiprogramming Systems (1970s - 1980s)

Network and Distributed Systems (1980s - 1990s)

Modern Operating Systems (1990s - Present)

Defining RTOS (Real-Time Operating System)

- An **RTOS** is a type of operating system designed to serve real-time applications that process data as it comes in, typically without buffering delays.
- Real-time systems are often used in critical applications where precise timing and predictability are essential (e.g., embedded systems, robotics, aerospace).

serve real-
time
applications

without
buffering
delays

Time critical
applications

Key Features:

- **Deterministic:** RTOS guarantees that tasks are executed within a defined time frame.
- **Minimal Latency:** The system reacts quickly to external events, often within milliseconds or microseconds.
- **Task Prioritization:** Critical tasks have higher priority and are executed first.
- **Example:** RTOS is commonly used in **medical devices** like pacemakers, where it must react to heart signals in real-time to ensure proper functionality.

Deterministic

**Minimal
Latency**

**Task
Prioritization**

The Scheduler in RTOS

- The **scheduler** is a core component of an RTOS, responsible for deciding the order and timing of tasks executed by the CPU. There are two primary types of schedulers:
 1. **Preemptive Scheduling:** The scheduler can interrupt tasks to assign the CPU to a higher-priority task. This ensures that the most critical tasks are handled immediately.
 - **Example:** In a car's anti-lock braking system (ABS), if the system detects a wheel lock, it preempts any less critical tasks to ensure immediate brake control.
 2. **Cooperative Scheduling:** Tasks voluntarily give up control of the CPU when they are done or enter a waiting state.
 - **Example:** In a simple thermostat, tasks cooperate to read the temperature sensor and adjust the heater periodically.

**Preemptive
Scheduling**

**Cooperative
Scheduling**

Types of Scheduling Algorithms

- **Rate Monotonic Scheduling (RMS):** Tasks with shorter periods have higher priority.
- **Earliest Deadline First (EDF):** Tasks are prioritized based on their deadlines, with closer deadlines getting higher priority.

**Rate Monotonic
Scheduling
(RMS)**

**Earliest
Deadline First
(EDF)**

Objects in RTOS

- In RTOS, **objects** are entities used for managing tasks and communication. Some common RTOS objects include:
- **Tasks/Threads:** Independent units of execution.
 - **Example:** In a robotic arm, one thread might control the movement while another thread handles sensors.
- **Semaphores:** Synchronization tools used to control access to shared resources.
 - **Example:** In an elevator system, a semaphore ensures that only one elevator moves at a time when using the same motor resource.
- **Message Queues:** Used for sending messages between tasks.
 - **Example:** In a drone, different tasks (e.g., navigation and camera control) communicate with each other through message queues.
- **Timers:** Objects that are used to trigger events after a specific time period.
 - **Example:** A sprinkler system might use a timer to start and stop watering at predefined intervals.

Tasks/Threads

Semaphores

Message
Queues

Timers

Services in RTOS

- An RTOS provides various **services** that aid in system operation and task management. Common services include:
- **Task Management:** Creating, deleting, suspending, and resuming tasks.
 - **Example:** A smart home system creates a task to monitor temperature and suspends it when no one is home.
- **Memory Management:** Allocating and de-allocating memory for tasks.
 - **Example:** A digital camera allocates memory for image buffers and releases it after processing is complete.
- **Inter-Process Communication (IPC):** Mechanisms like message queues, semaphores, or shared memory used to enable tasks to communicate.
 - **Example:** In an industrial robot, one task sends a message to another task to trigger the next action.
- **Device Drivers:** Managing and controlling hardware devices like sensors, actuators, and communication interfaces.
 - **Example:** An RTOS in an autonomous vehicle uses drivers to interface with LIDAR sensors for real-time obstacle detection.

**Task
Management**

**Memory
Management**

**Inter-Process
Communication**

Device Drivers

Characteristics of RTOS

- **Real-Time Constraints:** Tasks must meet strict deadlines. Tasks must be completed within the set time frame (often measured in milliseconds).
 - **Example:** In a missile guidance system, the RTOS must calculate the trajectory within milliseconds to adjust the missile's course.
- **Predictability and Reliability:** The system's behavior must be predictable, meaning tasks are completed consistently in a given time period.
 - **Example:** In an aircraft's flight control system, predictable response times are essential for stable flight.
- **Low Jitter:** The variation in the time taken to execute tasks (jitter) must be minimal.
 - **Example:** In video streaming, low jitter ensures smooth playback without sudden delays.
- **Minimal Resource Utilization:** RTOSes are designed to be lightweight, using minimal memory and CPU cycles.
 - **Example:** Embedded systems in wearable devices (e.g., smartwatches) use RTOS to conserve battery power while maintaining responsiveness.
- **Multi-threading with Prioritization:** RTOS supports concurrent task execution with varying priorities.
 - **Example:** In a medical ventilator, critical tasks (airflow regulation) get higher priority over non-critical tasks (display updates).
- **Fault Tolerance:** Many RTOS systems incorporate mechanisms for recovering from hardware or software failures.
 - **Example:** In a satellite, an RTOS handles failure by switching to a backup task or subsystem if one part fails.

Real-Time
Constraints

Predictability
and Reliability

Low Jitter

Minimal
Resource
Utilization

Multi-
threading with
Prioritization

Fault Tolerance

Unit-2 Part-2

- Defining a Task,
 - Tasks States and Scheduling,
 - Task Operations,
 - Structure,
 - Synchronization,
 - Communication and Concurrency.

Task Management in Real-Time Operating Systems (RTOS)

- **Defining a Task**
- A **Task** in an RTOS is an independent unit of execution that performs a specific function.
- Tasks are akin to threads or processes in a general-purpose operating system but are often more lightweight and deterministic to meet real-time requirements.
- Tasks typically run in the background, continuously awaiting triggers or conditions to perform specific actions.

Independent unit
of execution

More lightweight
and deterministic

Run in the
background,
continuously
awaiting

Key characteristics of a Task:

- **Priority:** Each task has a defined priority that helps the RTOS scheduler determine which task to execute next.
- **State:** Tasks transition between different states based on events like execution, waiting for resources, or being preempted by higher-priority tasks.
- **Execution context:** Each task has its own context, including registers, program counter, and stack.
- **Example:** In an autonomous drone, one task might control the motors while another task manages the sensors. Each task is independent but interacts through synchronization mechanisms.

Priority

State

**Execution
context**

Task States and Scheduling

- In RTOS, tasks can be in several distinct states that define their status in terms of execution:
 - **Ready State:** The task is ready to run and is waiting to be assigned CPU time by the scheduler.
 - **Running State:** The task is actively being executed by the CPU.
 - **Blocked State (Waiting):** The task is waiting for an external event, such as the availability of a resource or the completion of an I/O operation.
 - **Suspended State:** The task is not ready for execution and won't be considered by the scheduler until explicitly resumed.
- These states allow the system to efficiently manage multiple tasks and switch between them as necessary.

Ready State

Running State

Blocked State
(Waiting

Suspended
State

Scheduling

Scheduling: The RTOS uses scheduling algorithms to decide which task should run at a given time, based on task priority and system conditions. The most common scheduling strategies include:

1. **Priority-Based Scheduling:** The highest-priority task is executed first. If a higher-priority task becomes ready, the RTOS may preempt the currently running task.
 - **Example:** In an automotive braking system, a high-priority task responsible for emergency braking overrides lower-priority tasks like temperature monitoring.
2. **Round Robin Scheduling:** Each task is given a fixed time slice in a cyclic order. This approach is typically used when tasks have equal priority.
 - **Example:** In a simple data acquisition system, each sensor task might take turns gathering data within a set time window.
3. **Deadline-Based Scheduling:** Tasks with closer deadlines are given higher priority.
 - **Example:** In video streaming, tasks responsible for processing video frames with deadlines are given precedence to ensure smooth playback.

**Priority-Based
Scheduling**

**Round Robin
Scheduling**

**Deadline-
Based
Scheduling**

Task Operations

- RTOS provides services to manage tasks, including:
 1. **Task Creation:** A new task is created with a specified function, priority, and stack size.
 - **Example:** In a robotic arm, a task might be created to control the movement of each joint.
 2. **Task Deletion:** When a task is no longer needed, it can be deleted to free up system resources.
 - **Example:** After completing a specific sensor reading, a task might be deleted in an IoT device to conserve memory.
 3. **Task Suspension:** A task can be temporarily suspended, preventing it from running until it is explicitly resumed.
 - **Example:** A task controlling an idle machine can be suspended when no work is needed and resumed when a new job starts.
 4. **Task Resumption:** Suspended tasks can be resumed when necessary.
 - **Example:** The robotic arm task might be resumed once a new command for movement is issued.
 5. **Task Prioritization:** The priority of a task can be dynamically changed based on system needs.
 - **Example:** In a factory automation system, the priority of a task controlling emergency shutdown may be raised if critical conditions are detected.
 6. **Task Delay:** A task can be put to sleep for a specified amount of time before being placed back in the ready state.
 - **Example:** In a time-based system like a temperature monitoring device, a task might be delayed for a specific interval between readings.

Task Creation

Task Deletion

Task
Suspension

Task
Resumption

Task
Prioritization

Task Delay

Structure of a Task

- A task typically includes:
 1. **Task Function:** The code that defines the specific functionality the task performs.
 - **Example:** In a home automation system, a task function may control the lights based on sensor input.
 2. **Task Stack:** The memory space where a task stores its local variables, function call frames, and context.
 - **Example:** A task stack for a thermostat might store temperature readings and control commands.
 3. **Task Context:** The register values, program counter, and other hardware-specific information required to resume the task after a context switch.
 4. **Task Control Block (TCB):** Data structure that stores information about the task, such as its state, priority, and stack pointer.
 - **Example:** Each task in a drone control system would have a TCB to track its status and priority.

**Task
Function**

Task Stack

**Task
Context**

**Task Control
Block (TCB)**

Synchronization

- Tasks in an RTOS often need to synchronize with each other to share resources or coordinate actions. Common synchronization mechanisms include:
 - **Semaphores:** A signaling mechanism that controls access to shared resources by multiple tasks.
 - **Example:** In an RTOS controlling a printer, semaphores might be used to ensure only one task can send data to the printer at a time.
 - **Mutexes (Mutual Exclusion):** A mutex is similar to a semaphore but provides exclusive access to a resource, preventing race conditions.
 - **Example:** In a file system, a mutex ensures that only one task writes to a file at any given time.
 - **Event Flags:** Used for tasks to wait for specific events to occur.
 - **Example:** In a sensor network, a task may wait for an event flag indicating new data has arrived before processing it.
 - **Barriers:** A synchronization point where tasks must all reach before any can proceed.
 - **Example:** In parallel data processing, tasks might synchronize at a barrier to ensure that all threads complete their work before moving to the next step.

Semaphores

Mutexes

Event Flags

Barriers

Communication and Concurrency

- RTOS tasks must often communicate with each other, and this is usually done using mechanisms like:
 - **Message Queues:** Tasks communicate by sending and receiving messages through queues. Message queues ensure safe communication between tasks by managing access and order of delivery.
 - **Example:** In a smart home system, a task that controls the thermostat might send temperature updates to another task managing the air conditioning.
 - **Shared Memory:** Multiple tasks can share a portion of memory for faster data exchange. However, synchronization mechanisms like semaphores or mutexes are typically required to avoid conflicts.
 - **Example:** In a multimedia player, tasks that handle audio and video synchronization might share memory to store the media stream.
 - **Pipes or Streams:** Tasks send data through continuous streams, typically used in real-time data processing.
 - **Example:** In a sound-processing system, tasks might send audio data through a pipe to the output driver.
 - **Concurrency:** RTOS tasks often run concurrently, meaning multiple tasks execute seemingly in parallel. To manage this, the RTOS uses the scheduler to switch between tasks rapidly, creating the illusion of parallelism.
 - **Example:** In an autonomous vehicle, tasks controlling navigation, obstacle detection, and sensor monitoring might all run concurrently to ensure smooth operation.

Message
Queues

Shared
Memory

Pipes or
Streams

Concurrency

Unit-2 part-3

- Defining Semaphores
 - Operations and Use

Defining Semaphores

- A **semaphore** is a synchronization mechanism used in operating systems, including real-time operating systems (RTOS), to control access to shared resources by multiple tasks or threads.
- It helps in preventing race conditions and ensuring that only a specified number of tasks can access a shared resource simultaneously.

synchronization
primitive

control access
to shared
resources

multiple tasks
or threads

prevents race
conditions

Semaphores Types

- Semaphores are counters that track the availability of resources. They have two main types:
 - **Binary Semaphores:** These can only take values of 0 or 1, essentially acting as a lock for mutual exclusion (mutex). A binary semaphore ensures that only one task at a time can access a shared resource.
 - **Example:** In a printer management system, a binary semaphore is used to ensure that only one task sends a print job at a time.
 - **Counting Semaphores:** These can take non-negative integer values and are used to allow multiple instances of a resource to be available. A counting semaphore controls how many tasks can access a resource simultaneously.
 - **Example:** In a system with multiple I/O ports, a counting semaphore could be used to allow up to three tasks to access three ports at the same time.

**Binary
Semaphores**

**Counting
Semaphores**

Operations and Use

- Semaphores work by providing two key operations:
 - **1. Wait (P Operation or down)**
- This operation decreases the semaphore's value. If the value is greater than 0, the semaphore grants access to the task. If the value is 0 or less, the task is blocked and added to a waiting queue until the resource becomes available.
- **Steps:**
 - Decrement the semaphore value.
 - If the value is negative or 0, block the task.
 - If positive, allow access.
 - **Example:** A task trying to access a file would use the "Wait" operation on a semaphore. If the file is already being used (semaphore value is 0), the task will wait until the file becomes available.
 - **2. Signal (V Operation or up)**
- This operation increases the semaphore's value. If any tasks are waiting, one of them is unblocked and given access to the resource. Otherwise, the semaphore value is simply incremented.
- **Steps:**
 - Increment the semaphore value.
 - If any tasks are waiting (value was 0 or negative), unblock a task and allow it to proceed.
 - If no tasks are waiting, the semaphore value is simply increased.
 - **Example:** When a task finishes writing to a file, it signals the semaphore, indicating the file is now available for use by another task.

**Wait()-
decrement**

**Signal()-
increment**

Use Cases for Semaphores

- **Mutual Exclusion (Mutex)**
 - **Binary semaphores** are used to ensure that only one task can access a critical section of code or a resource at a time.
 - **Example:** In a banking application, a binary semaphore ensures that only one task can access and update an account balance at any given moment, preventing race conditions.
- **Resource Counting**
 - **Counting semaphores** are useful when you need to manage access to multiple instances of a resource (e.g., network connections, memory blocks).
 - **Example:** In a server managing multiple client connections, a counting semaphore with an initial value equal to the number of available connections ensures that no more than the allowed number of clients are connected at any time.
- **Task Synchronization**
 - Semaphores are used to synchronize tasks, ensuring that one task waits for another task to complete before proceeding.
 - **Example:** In a sensor system, a task that processes sensor data might wait for a semaphore indicating that new data is available before processing.
- **Producer-Consumer Problem**
 - Semaphores solve synchronization problems where one task produces data and another task consumes it, ensuring the consumer waits if there's no data and the producer waits if the buffer is full.
 - **Example:** In a real-time logging system, a counting semaphore ensures the producer writes log data to a buffer, and the consumer reads and processes it only when data is available.

Mutual Exclusion

Resource
Counting

Task
Synchronization

Producer-Consumer
Problem

Example: Semaphore in a Printing System

- Consider a system with two printers and three tasks that want to print documents.
- **Counting Semaphore Initialization:**
 - A counting semaphore is initialized with the value 2 (since there are two printers available).
- **Task 1 Wants to Print:**
 - Task 1 performs a "Wait" operation. The semaphore value decreases to 1, allowing Task 1 to use one of the printers.
- **Task 2 Wants to Print:**
 - Task 2 also performs a "Wait" operation, decreasing the semaphore value to 0. Task 2 gets the other available printer.
- **Task 3 Wants to Print:**
 - Task 3 performs a "Wait" operation, but since the semaphore value is 0, it has to wait until one of the printers becomes free.
- **Task 1 Finishes Printing:**
 - Task 1 performs a "Signal" operation, increasing the semaphore value to 1. Task 3, which is waiting, is now allowed to proceed and use the printer.

Unit-2 part-4

- Defining Message Queue,
 - States, Content,
 - Storage,
 - Operations and Use

Defining Message Queue

- A **message queue** is a communication mechanism used in real-time operating systems (RTOS) and other multitasking environments to allow tasks (or processes) to send and receive messages asynchronously.
- The messages are stored in the queue and can be accessed by the receiving task when it's ready, decoupling the producer and consumer tasks.
- This allows tasks to communicate without requiring direct synchronization or shared memory.
- A message queue helps coordinate data exchange between tasks and provides a buffer that holds messages until the receiving task retrieves them.

Asynchronous
communication

messages are
stored in the
queue

communicate
without direct
connection

coordinate data
exchange

States of a Message Queue

- A message queue can exist in various states, depending on its usage and the availability of messages:
 - **Empty:** The queue is empty when no messages are present. Any task trying to retrieve a message will be blocked or informed that no message is available.
 - **Example:** A logging system might have an empty queue initially until new log entries are created by tasks.
 - **Partially Full:** The queue has some messages, but it is not yet full. Producer tasks can continue sending messages, and consumer tasks can retrieve them.
 - **Example:** A real-time sensor system might have a partially full queue where some sensor readings are already waiting to be processed.
 - **Full:** The queue is full when it reaches its maximum capacity, and no further messages can be sent until the consumer task retrieves some.
 - **Example:** In an industrial monitoring system, if the message queue for alerts is full, the system might block any new alerts from being queued until some are processed.

Empty

Partially
Full

Full

Content of a Message Queue

- A message queue can hold messages in various formats, and the content structure depends on the specific system requirements. Each message typically includes:
 - **Message Data:** This is the main content or payload of the message. It can be anything, such as sensor readings, commands, status updates, or other relevant data.
 - **Example:** In a weather monitoring system, the message data might include temperature, humidity, and pressure readings from a sensor.
 - **Message Length:** This defines the size of the message. Some RTOS implementations might require that all messages in the queue be of fixed length, while others allow variable-length messages.
 - **Example:** Messages containing text or binary data can vary in size, depending on the content.
 - **Message Priority (optional):** Some message queues support priority levels for each message, which can affect the order in which messages are retrieved. Higher-priority messages are handled first, even if they arrive later.
 - **Example:** In a networked system, critical alerts about security breaches may be given higher priority compared to routine status updates.

**Message
Data**

**Message
Length**

**Message
Priority**

Storage of Messages

- Messages in a message queue are stored in memory, typically in one of two ways:
 - **First-In-First-Out (FIFO) Order:** Messages are retrieved in the order they were sent. This is the most common method of storage for message queues.
 - **Example:** In a system that processes video frames, each frame would be queued and processed in the order it was captured.
 - **Priority-Based Storage:** Some message queues allow messages to be stored based on priority, where higher-priority messages are retrieved before lower-priority ones.
 - **Example:** In a real-time alarm system, critical alarms may be placed at the front of the queue, even if they arrive after less important messages.
- The message queue is typically stored in **RAM** and managed by the RTOS, which tracks the current state, message contents, and the tasks waiting for messages.

**First-In-First-
Out (FIFO)
Order**

**Priority-Based
Storage**

Operations on a Message Queue

- The key operations provided by message queues include:

1. Send (Enqueue)

- This operation sends a message to the queue. If the queue is not full, the message is added, and the sending task continues execution. If the queue is full, the sending task may be blocked (waiting for space), return an error, or overwrite the oldest message, depending on the system configuration.

- **Steps:**

- Add the message to the queue if space is available.
- If the queue is full, handle the condition based on the system design (block, drop, or overwrite).
- **Example:** In an IoT system, a temperature sensor task sends a reading to the queue for processing by the central controller.

2. Receive (Dequeue)

- This operation retrieves the next message from the queue. If the queue is empty, the receiving task may be blocked (waiting for a message) or return an error.

- **Steps:**

- Remove the oldest (or highest-priority) message from the queue.
- If the queue is empty, handle the condition by blocking the task or returning an error.
- **Example:** A task in a weather station system retrieves the latest set of sensor readings from the message queue for analysis.

3. Peek

- Some RTOS message queues provide a "peek" operation that allows a task to check the content of the next message without removing it from the queue.
- **Example:** In a network router, a task might peek at the next packet to determine its destination before processing it.

Blocking and Non-Blocking Modes

- Message queues can operate in blocking or non-blocking modes:
- **Blocking Mode:** Tasks are blocked if the queue is full (for sending) or empty (for receiving) until the condition changes.
- **Non-Blocking Mode:** Tasks are not blocked and immediately return with an error if the queue is full or empty.
 - **Example:** In a non-blocking mode, if a task tries to send a message to a full queue, it can continue execution without waiting, knowing that the message was not sent.

send

receive

peek

Use of Message Queues

- Message queues are commonly used in systems that require inter-task communication and decoupling between producers and consumers. Here are some typical use cases:

1. Task Communication

- Message queues allow tasks to communicate with each other without needing direct interaction. This is particularly useful in RTOS environments where tasks operate independently and need to pass data asynchronously.
- **Example:** In a robotic system, one task might monitor sensors and send position data to another task that controls movement.

2. Event Notification

- Message queues can be used to notify tasks of certain events. A task waiting for a specific event will block until a message related to that event is placed in the queue.
- **Example:** In a mobile phone, a message queue might be used to signal the arrival of an incoming call or SMS to the main task responsible for handling user interactions.

3. Producer-Consumer Problem

- In many systems, one task produces data (e.g., sensor readings), while another task consumes it (e.g., data processing). A message queue provides a buffer where data is stored temporarily until the consumer task is ready to process it.
- **Example:** In an audio streaming system, one task reads audio data from a file and places it in a message queue, while another task retrieves the data for playback.

4. Load Balancing

- In some systems, message queues are used for load balancing between tasks. If a task has multiple consumers, a message queue allows it to distribute work evenly across available resources.
- **Example:** In a web server, incoming requests are placed in a queue and processed by worker tasks, ensuring that each worker gets an equal share of the load.

5. Priority-Based Task Coordination

- When certain messages (or events) are more critical than others, message queues with priority levels ensure that high-priority messages are processed first.
- **Example:** In a real-time security system, an intruder alert message may be prioritized over routine system status messages.